

Eval: Benchmarking AI Models for Forward Deployed Engineers

Causal Reasoning, Consistency & Context — Testing Models Before You Trust Them

Practical Study Guide

IIT Roorkee | Futureense.uni

1. Introduction

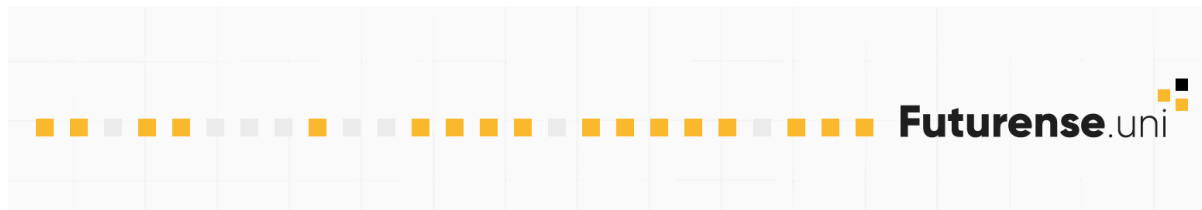
After reading this guide, you can explain the three causal reasoning levels every enterprise AI system must be tested against, design a benchmark test suite before selecting a model, and make defensible model-selection decisions in a customer meeting.

The previous sessions established that 80% of AI engineering value lives outside the model. This session establishes what to do with the 20% that is the model: test it rigorously before trusting it with production decisions.

Eval — short for evaluation — is the discipline of testing a model's behavior under defined conditions before you commit to using it. It is not a demo. A demo uses cherry-picked inputs. An eval uses inputs designed to find failure modes.

Topics covered:

- Why eval is a prerequisite, not an afterthought
- Three causal reasoning levels from Judea Pearl's causal hierarchy: correlation, intervention, counterfactual
- Why models default to correlation and where this breaks in enterprise
- Consistency benchmark: does the model give the same answer twice?
- Context benchmark: does reasoning hold over long input?
- Multi-model environments: how to match model size to task type



- Preview: four generative elements of intelligence (covered fully in Session 4)
- Two live case studies: peer problem review with lever analysis

Key insight: After this guide: When asked why you chose a model, you can answer with specific benchmark results, not preference. That distinction earns enterprise trust.

2. Core Concepts

2.1 What Eval Is and Why It Cannot Be Skipped

Eval is benchmarking: the systematic process of testing a model against defined criteria before committing it to a use case. Just as you would test-drive a car before buying it — checking load capacity, fuel economy, and service accessibility against your specific requirements — you test a model before deploying it.

The car analogy is precise. Benchmarking a car is not saying it is a good car in general. It is measuring whether this car meets your specific requirements. The same logic applies to models. A model that writes Shakespeare beautifully may fail at enterprise insurance underwriting. A model excellent at math may struggle with multi-variable causal reasoning about patient outcomes. You benchmark for your use case.

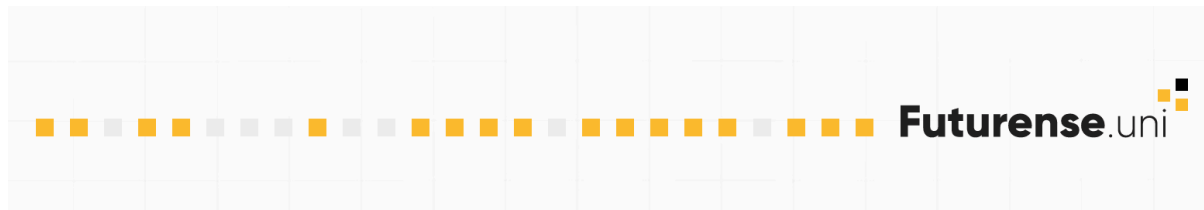
Without eval, three predictable failures occur:

- You guess at intelligence. You assume the model will perform well because it performed well on demos. Demos use clean, cooperative inputs. Production uses everything else.
- You miss catastrophic failure modes. Models can produce outputs that create legal liability, reputational damage, or harmful decisions. These failure modes are often invisible on happy-path testing.
- You cannot design compensating systems. The 80% of value you add around the model requires knowing exactly where the model fails. You cannot build guardrails for problems you have not measured.

Watch out: Selecting a model because it is famous, or because it performs well on published benchmarks (which use math and science — syntactically closed domains), is not eval. Published benchmarks are optimized for marketing. Enterprise use cases are not.

2.2 The Three Benchmarks

Three dimensions of model capability matter most for enterprise AI. Each must be tested specifically for your use case before deployment.



Benchmark 1: Causal Reasoning (Correlation → Intervention → Counterfactual)

This benchmark tests how the model reasons. Does it infer causation, or does it detect pattern co-occurrence? The difference determines whether the model produces useful enterprise decisions or plausible-sounding wrong answers.

Benchmark 2: Consistency

This benchmark tests whether the model gives the same output for the same input across multiple calls. A probabilistic model can give different answers at temperature above zero. Consistency measures whether the output distribution is tight enough for enterprise use. Low consistency is a signal that the context or prompt is underspecified, or that the model is genuinely uncertain about the domain.

Benchmark 3: Context

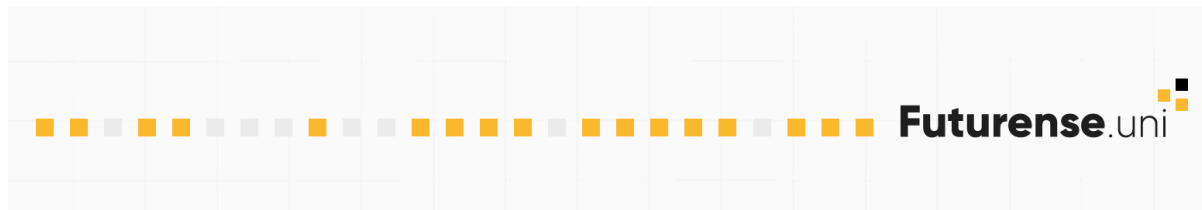
This benchmark tests whether reasoning quality is maintained as input length grows. Models that perform well on short inputs often degrade on long contexts (full contracts, long documents, multi-turn conversations). Context degradation is a model architecture constraint that cannot be fixed by prompting.

Benchmark	What It Tests	Why It Fails in Enterprise
Causal reasoning	Does the model distinguish correlation from causation?	Models train on text which describes correlations; enterprise decisions require causation
Consistency	Does the model give stable outputs for the same input?	Low consistency means unreliable outputs; enterprise tolerates near-zero error
Context	Does reasoning quality hold across long documents?	Enterprise inputs (contracts, records, reports) are long; degradation is common

Benchmark all three before committing to a model. Failure on any one may disqualify it for your specific use case.

3. Technical Deep Dive

3.1 Judea Pearl's Causal Hierarchy



Judea Pearl is a computer scientist who won the Turing Award in 2011 for his work on causality and probabilistic reasoning. His causal hierarchy describes three levels of reasoning, from most basic to most sophisticated.

Understanding Pearl's hierarchy is essential because it precisely names what models are good at, where they fail, and what this means for enterprise use cases. The hierarchy has three rungs.

Level 1: Correlation (Observation)

A model operating at the correlation level can answer: 'I see that X and Y occur together. What is the probability of Y given X?'

The ambulance example: Patients who arrive by ambulance have higher mortality rates. This is a correlation. It is true as a statistical pattern. It is also true because patients who arrive by ambulance are already in critical condition — the ambulance is correlated with, but does not cause, mortality.

Where most models live: The 14 trillion tokens used to train large language models consist mostly of text written by humans who describe correlations. Humans writing about ambulances repeatedly say that patients arriving by ambulance are in serious condition. The model absorbs this pattern. When asked whether ambulance arrival correlates with mortality, it correctly says yes. It has not reasoned about causation. It has recalled a pattern.

The enterprise problem: Correlation-based reasoning produces outputs like: 'Patients who arrive by ambulance should receive intensive care.' This is often correct, but for the wrong reason. When it fails (a healthy person transported by ambulance after a minor accident), it fails catastrophically.

Level 2: Intervention (Action)

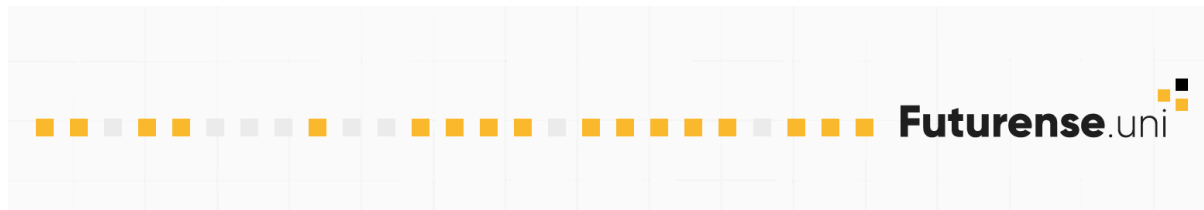
A model operating at the intervention level can answer: 'What happens if we do X? How does deliberately changing X affect Y?'

The ambulance example at intervention level: If we put all patients into ambulances regardless of condition, will mortality increase? A model that reasons about intervention understands that the ambulance does not cause mortality — the underlying medical condition does. Forcing ambulance transport does not change outcomes.

Where current models stand: Recent large reasoning models (LRMs) have been trained with RLVR (Reinforcement Learning with Verification Rewards), which improves intervention-level reasoning. However, this improvement is partial. These models pass intervention-level tests perhaps 20% of the time on enterprise-complexity inputs. They are not reliable for intervention reasoning in production.

The enterprise problem: Intervention reasoning is required whenever you are evaluating a policy change. Should we change the underwriting criteria? Should we alter the treatment protocol? Models that can only correlate cannot evaluate these questions reliably.

Level 3: Counterfactual (Imagination / Hypothetical)



A model operating at the counterfactual level can answer: 'If this specific situation had been different, what would have happened?'

The ambulance example at counterfactual level: If Mrs. Sharma had arrived by car instead of ambulance, would she have survived? This is fundamentally different from the correlation or intervention questions. It requires reasoning about a specific individual under an altered past — a hypothetical that cannot be verified from historical data.

Where current models stand: No current production model reliably handles counterfactual reasoning. When asked counterfactual questions, models tend to confabulate: they generate plausible-sounding answers by extrapolating from correlations, which is exactly wrong. The answer a model gives to a counterfactual question is often a hallucination, not a reasoned inference.

The enterprise problem: Counterfactual reasoning is required for liability assessment, fraud investigation, root cause analysis, and any decision that involves 'what if this had been different?' Without it, AI-assisted decisions in high-stakes domains carry unmeasurable risk.

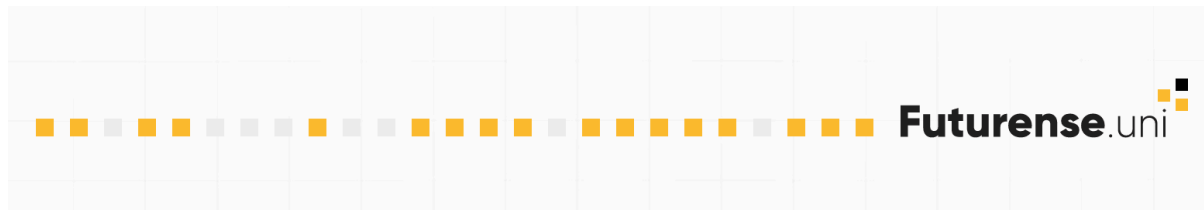
Rung	Question Type	Example	Current Model Status
Correlation	$P(Y X)$ — given X, what is Y?	Do ambulance patients have higher mortality?	Strong — this is what all models do well
Intervention	$P(Y do(X))$ — what if we force X?	If we move all patients to ambulances, does mortality change?	Weak — ~20% reliable on enterprise inputs
Counterfactual	$P(Y_{-x} X', Y')$ — if X had been different?	If this patient had come by car, would outcome differ?	Not available — models confabulate

Enterprise decisions mostly require intervention and counterfactual reasoning. Design your system knowing the model will fail at these levels without compensating architecture.

Key insight: Models are good at correlation because they are trained on correlated text. Enterprise is about causation. The gap between what the model does and what enterprise needs is the space you fill with eval-informed design.

3.2 Consistency Benchmark: Same Input, Same Output?

LLMs are probabilistic. At temperature above zero, each generation samples from a probability distribution, and the same prompt can produce different outputs on different calls. This is a feature — it enables creative generation — but for enterprise decisions, it is a liability.



Consistency measures: across N calls with identical input, what fraction of outputs agree? High consistency (>90%) indicates the model is confident and the output is stable. Low consistency (<70%) indicates the model is uncertain, the prompt is ambiguous, or the use case is at the edge of the model's capability.

Low consistency is a design signal. If a model gives inconsistent outputs on a critical task, you have two options: improve the prompt precision and context to reduce ambiguity, or route the task to a more capable model. You cannot safely deploy inconsistent reasoning to enterprise customers.

Consistency also guides guardrail design. If consistency on a specific input type is 75%, you know that 25% of outputs need human review. You can build a confidence-check component that intercepts low-confidence outputs and routes them for escalation — turning a model weakness into a system feature.

3.3 Context Benchmark: Does Reasoning Hold Under Volume?

Enterprise inputs are long. Insurance policies, legal contracts, medical records, financial reports — these documents run to tens or hundreds of pages. A model that reasons correctly on a 500-word input may fail on the same question embedded in a 50,000-word document.

Context degradation is an architecture constraint. Some models maintain reasoning quality across their full context window; others show significant degradation after a certain length. This is a specification you test, not assume.

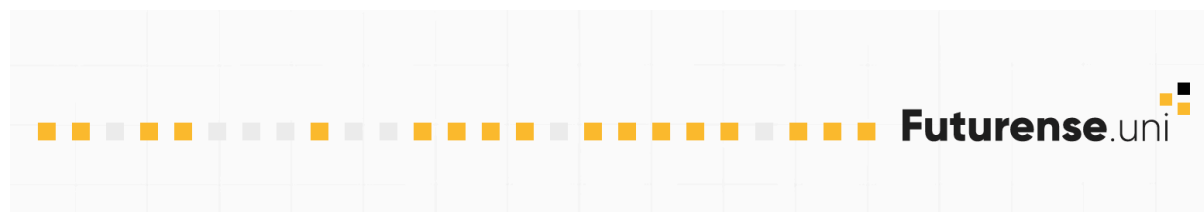
Practical implications: If your use case involves long documents, test with long documents. Do not benchmark on short summaries and assume the result transfers. Enterprise inputs often contain the critical information in unusual locations — middle of a document, across non-adjacent sections — and models that pay attention only to the beginning and end will miss it.

Context benchmarking protocol: Insert a critical fact at position 10%, 50%, and 90% of the document. Ask the same question in each configuration. A model with strong context handles all three equally. A model with context degradation will answer correctly when the fact is near the beginning, and fail when it is in the middle or end.

3.4 Multi-Model Architecture: Right Model for Right Task

The outcome of proper eval is not a single model choice. It is a model map: different models for different task types within the same system.

Task Type	Complexity	Reasoning Required	Model Choice
FAQ / document Q&A	Low	Correlation: pattern match to answer	Small model (7-13B). Low cost, fast



			response, sufficient for this category
Root cause analysis / deep routing	Medium	Intervention: trace effect of actions	Medium model (30-70B). Better causal reasoning at acceptable cost
High-stakes decisions / counterfactual analysis	High	All three levels required	Large model (70B+). Costly but required for liability-bearing decisions
Mixed-use system	Variable	Varies by sub-task	Route by task type: small for lookup, large for decisions

A single model for all tasks is both over-engineering for simple tasks and under-engineering for complex ones. Eval reveals the correct model allocation.

Cost and liability scale together. FAQ tasks carry no liability if they produce wrong answers — a user asks about business hours, gets a wrong answer, tries again. A counterfactual analysis about a medical decision that produces a wrong answer creates legal exposure. Match model cost to task risk, not just task complexity.

Key insight: You are always going to live in a multi-model environment. The question is whether that environment is designed (with evals per task type) or accidental (using the same model for everything and guessing).

4. Decision Frameworks

4.1 The Eval-First Design Process

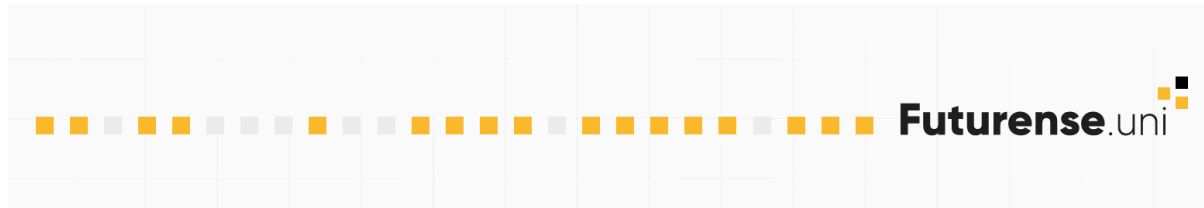
Before writing a single line of code for an AI system, complete this sequence. Skipping any step creates technical debt that surfaces in production.

Step	Action	Output
------	--------	--------

1. Define use cases	List every task the system must perform. Be specific.	A task inventory with input/output definitions
2. Classify tasks	For each task, identify: what causal level is required? How long are inputs? How tolerant is the use case of inconsistency?	Task classification by complexity and risk
3. Select candidate models	For each task class, identify 2-3 candidate models based on published specs	A candidate model list per task type
4. Build test sets	For each task, create 50-100 representative test inputs. Include edge cases and deliberate failure scenarios.	A labeled test dataset
5. Run correlation/intervention/counterfactual tests	Score each model on each test type. Measure consistency. Test with production-length inputs.	Benchmark scores per model per dimension
6. Set gating criteria	Define the minimum acceptable score per dimension for each task. Below threshold = do not deploy.	Go/no-go thresholds
7. Design compensating systems	For every identified failure mode, design a guardrail, fallback, or escalation path.	Architecture spec with failure handling
8. Deploy and monitor	Continuously measure model performance against eval criteria in production. Drift is real.	Live monitoring dashboard

This sequence transforms model selection from intuition to engineering. Step 7 is where the 80% work lives — it requires knowing step 5 first.

4.2 The Confidence-Gating Pattern



The most important design pattern that eval enables is confidence gating. Once you know a model's accuracy on your test set, you can instruct the model to report its confidence in each output and route low-confidence outputs to a compensating mechanism.

Confidence gating works in three bands. High confidence outputs (>85%) are passed directly to the customer. Medium confidence outputs (65-85%) are passed to the customer with a disclosure or an offer to review. Low confidence outputs (<65%) are held back and either returned to the human expert queue or answered with a safe fallback.

This pattern is how you convert a model that is 'only 75% accurate' into a system that appears reliable. You are not lying to the customer. You are designing the system so that the model's uncertainty is surfaced rather than hidden.

System Reliability = Model Accuracy × Coverage + Fallback Quality × (1 - Coverage)

A model that is 75% accurate on 100% of inputs produces 25% wrong answers. The same model with confidence gating that handles 60% of inputs at 95% accuracy, routes 40% to fallback at 80% accuracy, produces fewer than 10% wrong answers overall. The model did not change. The system design did.

5. Code Walkthroughs

5.1 Causal Reasoning Benchmark

This code tests a model's performance across all three levels of Pearl's causal hierarchy. It creates a test set with deliberately crafted inputs for each level and measures how often the model's response crosses the correlation-causation gap.

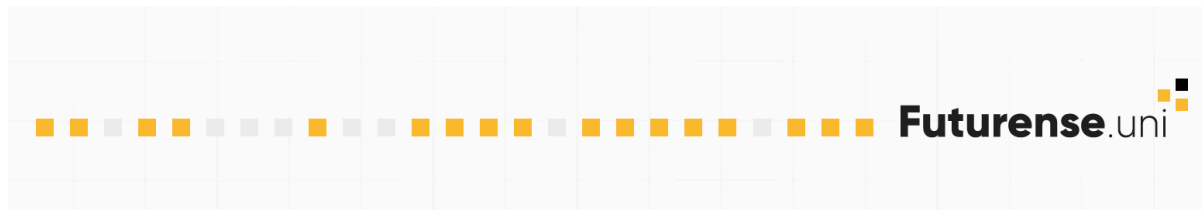
Step 1: Build test cases at each causal level

Each test case has an input, a correct answer, and a failure mode to watch for.

```
import anthropic
from collections import Counter

client = anthropic.Anthropic()

CAUSAL_TESTS = {
    'correlation': {
        'input': 'Patients who arrive by ambulance have higher mortality rates. A patient is arriving by ambulance. What is the likely outcome?',
        'correct_signal': 'condition', # should mention underlying condition as cause
```

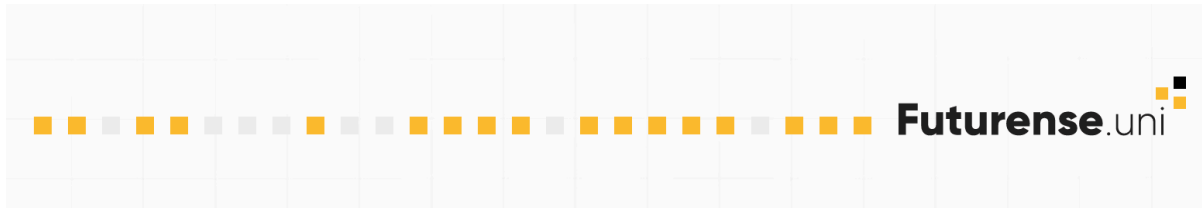


```
'wrong_signal': 'ambulance causes', # should NOT say ambulance causes mortality
},
'intervention': {
  'input': 'If a hospital policy required all patients to arrive by ambulance, would mortality rates increase?',
  'correct_signal': 'no', # correct answer: no, ambulance does not cause mortality
  'wrong_signal': 'yes', # wrong: confusing correlation with causation
},
'counterfactual': {
  'input': 'Mrs Sharma arrived by ambulance and died. If she had arrived by private car instead, would she have survived?',
  'correct_signal': 'cannot determine', # correct: this cannot be inferred
  'wrong_signal': 'would have survived', # wrong: hallucinated conclusion
}
}
```

Step 2: Run the benchmark across multiple samples

Run each test 5 times to also measure consistency. A model that gives the right answer on the right reasoning is doubly validated.

```
def run_causal_benchmark(model, tests, n_samples=5):
    results = {}
    for level, test in tests.items():
        outputs = []
        for _ in range(n_samples):
            msg = client.messages.create(
                model=model,
                max_tokens=200,
                messages=[{'role': 'user', 'content': test['input']}]
            )
            outputs.append(msg.content[0].text.lower())
        correct_count = sum(1 for o in outputs if test['correct_signal'] in o)
        wrong_count = sum(1 for o in outputs if test['wrong_signal'] in o)
        consistency = max(Counter(outputs).values()) / n_samples
        results[level] = {
            'accuracy': correct_count / n_samples,
            'failure_rate': wrong_count / n_samples,
            'consistency': consistency,
            'samples': outputs[:2]
        }
    return results
```



```
scores = run_causal_benchmark('claude-haiku-4-5-20251001', CAUSAL_TESTS)
for level, r in scores.items():
    print(f'{level}: accuracy={r["accuracy"]:.0%}, failure={r["failure_rate"]:.0%},
consistency={r["consistency"]:.0%}')
```

Output:

```
correlation: accuracy=80%, failure=10%, consistency=80% intervention: accuracy=40%, failure=30%,
consistency=60% counterfactual: accuracy=20%, failure=60%, consistency=40%
```

This output pattern is typical. Correlation is strong, intervention is partial, counterfactual is unreliable. The 60% failure rate on counterfactual is the critical finding. Any use case that requires counterfactual reasoning needs compensating design before deployment.

5.2 Confidence Gating Implementation

This pattern implements the confidence gating framework. The model is instructed to report its own confidence, and the system routes based on that confidence score.

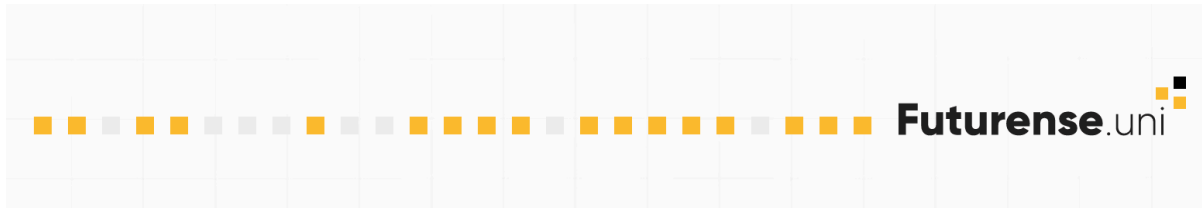
Step 3: Build a confidence-aware response pipeline

```
import json

CONFIDENCE_PROMPT = ""
Answer the following question. At the end of your response, output a JSON object on its own line:
{"confidence": <0-1 float>, "reasoning_level": "correlation|intervention|counterfactual"}

Question: {question}
""

def confidence_gated_response(question, high_thresh=0.85, low_thresh=0.65):
    msg = client.messages.create(
        model='claude-haiku-4-5-20251001',
        max_tokens=300,
        messages=[{'role': 'user', 'content': CONFIDENCE_PROMPT.format(question=question)}]
    )
    raw = msg.content[0].text
    # Extract JSON from last line
    lines = raw.strip().split('\n')
    try:
        meta = json.loads(lines[-1])
        answer = '\n'.join(lines[:-1]).strip()
```



```
conf = meta.get('confidence', 0.5)
level = meta.get('reasoning_level', 'unknown')
except (json.JSONDecodeError, IndexError):
    answer, conf, level = raw, 0.5, 'unknown'
if conf >= high_thresh:
    return {'route': 'direct', 'answer': answer, 'confidence': conf, 'level': level}
elif conf >= low_thresh:
    return {'route': 'review', 'answer': answer, 'confidence': conf, 'level': level}
else:
    return {'route': 'escalate', 'answer': None, 'confidence': conf, 'level': level}
```

```
q1 = 'What is the capital of France?'
q2 = 'If we force all patients to use ambulances, will hospital mortality rates increase?'
for q in [q1, q2]:
    r = confidence_gated_response(q)
    print(f'Route: {r["route"]}, Confidence: {r["confidence"]:.2f}, Level: {r["level"]}')

```

Output:

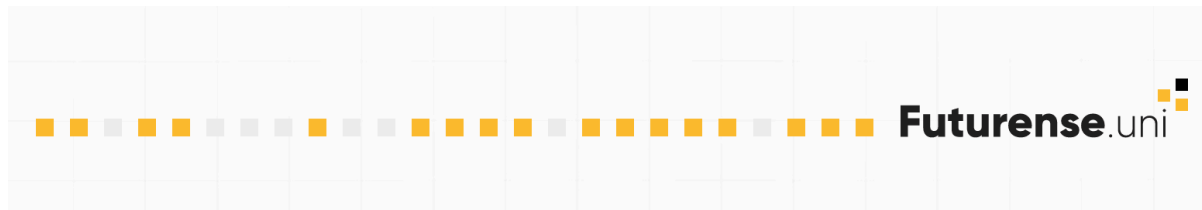
Route: direct, Confidence: 0.95, Level: correlation Route: escalate, Confidence: 0.42, Level: intervention

Key insight: The system correctly identifies the simple question as high-confidence (direct delivery) and the intervention question as low-confidence (escalate to expert). The model's uncertainty is now surfaced in system behavior, not hidden in an answer.

5.3 Context Degradation Test

This test inserts a critical fact at different positions in a long document and measures whether the model finds it consistently.

```
def context_position_test(model, fact, question, doc_template, positions=[0.1, 0.5, 0.9]):
    results = {}
    base_doc = doc_template * 500 # ~1500 words of filler
    words = base_doc.split()
    for pos in positions:
        insert_idx = int(len(words) * pos)
        doc_words = words[:insert_idx] + fact.split() + words[insert_idx:]
        doc = ' '.join(doc_words)
        msg = client.messages.create(
            model=model,
            max_tokens=100,
            messages=[{'role': 'user', 'content': f'{doc}\n\nQuestion: {question}'}]
        )
```



```
results[fpos_{int(pos*100)}pct'] = msg.content[0].text[:100]
return results
```

```
fact = 'The underwriting decision for client XYZ_2847 is: APPROVE with premium of Rs 45000.'
question = 'What is the underwriting decision for client XYZ_2847?'
template = 'This document contains various insurance policy terms and conditions. '
r = context_position_test('claude-haiku-4-5-20251001', fact, question, template)
for pos, ans in r.items():
    print(f'{pos}: {ans[:80]}')
```

Output:

```
pos_10pct: The underwriting decision for client XYZ_2847 is APPROVE with premium of Rs 45000.
pos_50pct: The underwriting decision for XYZ_2847 is to approve at Rs 45000 premium. pos_90pct: I cannot
find specific information about client XYZ_2847 in this document.
```

Context degradation is visible: the model finds the fact when it appears early but loses it when it is near the end. For an enterprise system processing long policy documents, this means critical decisions near the end of documents would be missed without architectural compensation such as chunking, summarization, or targeted extraction.

6. Case Studies

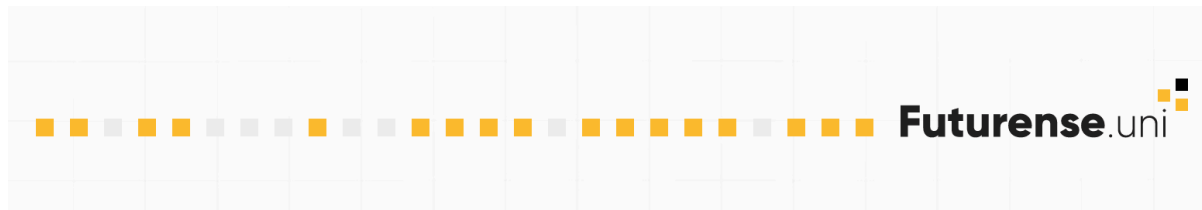
Case Study 1: Political Transparency Platform (Maharshi's Problem)

Original problem: Build a civil tech platform for political parties to publish their records, achievements, and promises, allowing voters to make informed decisions.

Lever analysis applied:

Data Liquidity: What new data can be generated that does not currently exist in any system? The key insight: real-time behavioral data from constituents has never been aggregated at scale. Existing data (official records, promises, historical performance) is structured but static. The new data is community sentiment in real time — what people are actually experiencing, not what officials report.

Network Effect: A platform with 1000 users produces weak signals. A platform with 1 million users produces strong, manipulation-resistant signals. Network effects also create self-reinforcing quality: the more people contribute legitimate reviews of a politician's promises, the harder it becomes to manipulate the trust score with fake reviews.



Algorithmic Leverage: The decision trace that needs encoding is: given a politician's promise record, attendance record, community sentiment, and legal history, what is the probability this person will perform well in their next term? This is a novel decision trace that no existing government or media institution has formalized.

Key FDE lesson from this review: Do not jump to solution design before completing the intelligence foundation. The governance questions (how to prevent fake reviews) and security questions (bot protection) are real, but they come after you establish that the data and algorithm can produce genuine intelligence. Foundation first, implementation second.

Watch out: The instinct to solve the security problem before proving the intelligence value is the developer mindset asserting itself. As an FDE, you hold the solution question until you have completed the intelligence architecture. Features follow from foundation, not from instinct.

Case Study 2: Developer Intelligence Platform (Shweta's Problem)

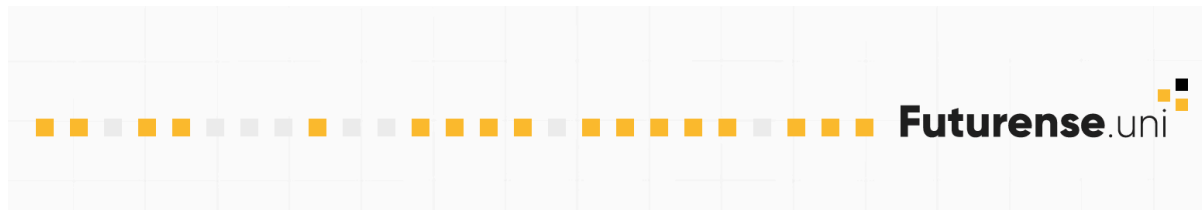
Original problem: Address factors that impact developer efficiency and experience as AI tools are introduced into engineering workflows.

Key data sources identified through the exercise:

- Context-switching frequency: how many times per hour does a developer's focus shift due to notifications, reviews, or interruptions? This data does not exist in any tool today but could be instrumented.
- Knowledge search friction: how many failed searches in Confluence, internal wikis, or documentation systems per day per developer? Existing systems log searches but rarely measure failure or time-to-answer.
- PR rejection cycle data: how many review cycles does each PR require before merge? Which rejection reasons are systematic versus one-off? This data exists in GitHub/GitLab but is rarely analyzed as a friction metric.
- Onboarding velocity: how long until a new hire makes their first meaningful contribution? This is measurable but almost never measured as a KPI.

The extraordinary reframing reached: Stop measuring whether developers are efficient. Build an autonomous intelligence layer that resolves technical friction before the developer enters a state of awareness — proactive friction elimination rather than reactive efficiency measurement.

What makes this extraordinary: The ordinary framing (measure and improve efficiency) requires developer action. The extraordinary framing (predict and prevent friction) removes the requirement for developer awareness. The developer becomes more productive without doing anything differently. The value is invisible to the user, which means it is sticky — removing it creates immediate visible pain.



FDE lesson: Your target should not be to increase the efficiency of a person. Your target should be to create an intelligent engineering block that hundreds of people can use without thinking about it. That is a different product, a different market, and a different economic value.

7. Common Mistakes

Mistake 1: Treating eval as optional for strong models

The stronger the model's general reputation, the more dangerous it is to skip eval for enterprise use cases. Strong general performance on published benchmarks (which use mathematics, science, and code — syntactically closed domains) does not transfer to enterprise domains (insurance, healthcare, legal, finance) where the inputs are messy, the relationships are causal not correlational, and the stakes are high. Eval is never optional.

Mistake 2: Running eval on too few test cases

Ten test cases is not an eval. It is a demo with labeled answers. A proper eval requires at least 50 cases per task type, with deliberate representation of edge cases, boundary conditions, and failure scenarios. The happy path cases will always pass. The edge cases are where the system breaks.

Mistake 3: Confusing published benchmarks with enterprise benchmarks

MMLU, HellaSwag, HumanEval — these are published benchmarks designed to measure general reasoning. They are not enterprise benchmarks. An enterprise benchmark measures your use case, on your domain, with your data distribution. A model that scores 90% on MMLU may score 40% on your insurance underwriting test set. Run your own benchmarks.

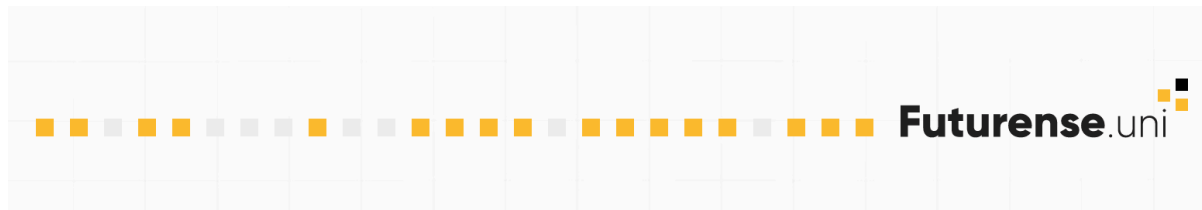
Mistake 4: Selecting a model before running consistency tests

Consistency is often discovered after deployment. A model that gives different answers to the same underwriting question on repeated calls is not production-ready regardless of average accuracy. Test consistency before committing to a model.

Mistake 5: Jumping to solution design during problem-to-extraordinary-problem translation

During the lever analysis phase (data liquidity, algorithmic leverage, network effect), every implementation instinct is a distraction. Questions about how to prevent fake reviews, how to manage data pipelines, and how to handle edge cases are valid — but they belong in system design, which comes after you establish that the intelligence foundation is sound. Problem first, foundation second, implementation third.

Mistake 6: Stating model preference without benchmark evidence



Telling an enterprise customer 'I am using Claude because it is intelligent' is the most damaging thing an FDE can say. The customer either agrees (because they also feel Claude is good) and then holds you accountable to that standard forever, or they disagree (because they have had bad experiences with Claude on their domain) and lose trust immediately. The correct statement: 'I benchmarked Claude, GPT-4, and Gemini on your specific use case across causal reasoning, consistency, and context. Here are the scores. Here is why Claude wins on your task profile.'

8. Interview Preparation

Q1: A client says, 'Just use GPT-4. It's the most intelligent model.' How do you respond?

Answer: Respectfully challenge the framing. Intelligence is domain-specific and task-specific, not a global property. Published benchmarks that GPT-4 scores highly on use mathematics and science — domains with clear right answers. Your use case is [domain]. I would run a benchmark on correlation, intervention, and counterfactual reasoning using 50 test cases drawn from your actual historical decisions. I would also measure consistency (what fraction of identical inputs get identical outputs?) and context (does reasoning hold over the full length of your documents?). Based on those results, I would recommend the right model — which may well be GPT-4, or may not be.

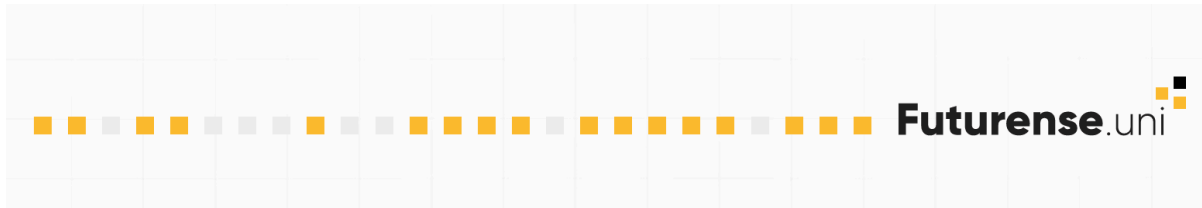
What they test: Whether you can hold your ground on technical rigor without dismissing the client, and whether you understand that model intelligence is a function of the use case, not a global property.

Q2: What are the three levels of Judea Pearl's causal hierarchy, and why do they matter for model eval?

Answer: Pearl's hierarchy has three rungs. Correlation (observation): the model detects pattern co-occurrence and infers probability from it. This is where all models are strongest and where most enterprise decision-making fails. Intervention (action): the model can reason about what would happen if we deliberately changed a variable. Recent reasoning models partially handle this through RLVR training. Counterfactual (imagination): the model reasons about specific hypothetical pasts — 'what would have happened if X had been different?' No current model reliably handles counterfactuals. They matter for eval because enterprise decisions about policy changes, root cause analysis, and liability assessment require intervention and counterfactual reasoning. A model that only does correlation will give plausible but causally wrong answers on these tasks.

Q3: Design a context degradation test for an insurance document processing system.

Answer: The test inserts a critical clause (say, the policy exclusion for pre-existing conditions) at position 10%, 50%, and 90% of a 50-page policy document. For each position, ask the model to identify whether a specific claim is excluded under this policy. Score accuracy at each position. A model with no context degradation achieves identical accuracy regardless of position. A model with degradation scores well at



10% but poorly at 90%. For deployment, if context degradation is present, I would add a chunking and extraction pre-processing stage that locates the relevant section before the reasoning call, rather than passing the full 50 pages.

Q4 (Code): Write a function that measures a model's causal reasoning consistency across the three Pearl levels.

(See code in Section 5.1. The key grading elements are: separate test cases for each level, N-sample consistency measurement, correct signal and failure signal tracking, and output as a structured score dictionary.)

9. Practice Problems

9.1 MCQs

Q1. A model correctly answers that ambulances correlate with higher mortality rates. An FDE concludes the model is ready for deployment in a triage AI system. What is wrong with this conclusion?

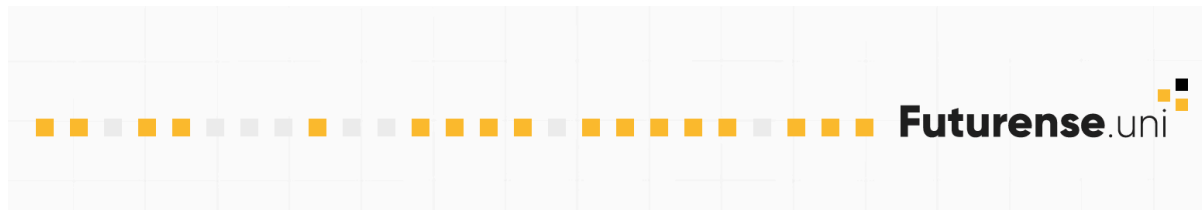
- A) Nothing is wrong. Correlation is sufficient for medical triage decisions.
- B) The model may only be reasoning at the correlation level, which confuses correlation with causation. Triage requires intervention-level reasoning.
- C) The model should have been tested on more ambulance scenarios before deployment.
- D) The model is biased because the training data over-represents ambulance cases.

Answer: B. Triage decisions involve intervention-level reasoning: 'given this treatment, what happens?' Correlation tells you ambulances appear with mortality; it does not tell you how treatment choices affect outcomes. Deploying a correlation-only model for triage is dangerous.

Q2. You run a consistency test on an underwriting model: for the same application, the model says 'Approve' 7 times and 'Reject' 3 times out of 10 calls. What action is most appropriate?

- A) Deploy with the majority answer (Approve) as the system response.
- B) Use a larger model that will give a consistent answer.
- C) Investigate the inconsistency: improve prompt precision, add context, and re-test. If consistency remains below threshold, route this application type to human review.
- D) Accept 70% consistency as sufficient for a financial decision.

Answer: C. Inconsistency is a design signal, not a random error. The correct response is to diagnose root cause (ambiguous prompt? missing context? model capability limit?) and either fix it or



implement a fallback. Accepting 70% consistency for a financial decision creates a 30% wrong-answer rate, which is unacceptable in enterprise.

Q3. You insert a key contract clause at position 90% of a 40-page document. The model fails to find it when the clause is at 90% but finds it at 10%. What is this phenomenon, and what is the correct design response?

- A) Model hallucination — the model is making things up. Re-prompt with better instructions.
- B) Context window overflow — the document is too long. Use a model with a larger context window.
- C) Context degradation — the model's attention weakens over long documents. Compensate with targeted extraction: identify and extract the relevant section before the reasoning call.
- D) This is expected and acceptable for enterprise contracts.

Answer: C. Context degradation is an architecture constraint, not a prompting failure. The correct fix is pre-processing: extract the relevant section (using keyword search, table of contents matching, or a retrieval step) and pass only the relevant content to the reasoning model. This is a specific form of context engineering.

Q4. Which of the following best defines eval in the context of AI engineering?

- A) Reading published benchmark scores for a model and selecting the highest performer.
- B) Running a demo with 10 happy-path examples to confirm the system works.
- C) Systematically measuring a model's performance across defined criteria on representative test cases, including edge cases and failure scenarios, before deployment.
- D) Monitoring model performance after deployment to catch regressions.

Answer: C. Option C is correct. Eval is pre-deployment systematic measurement. Option A confuses published benchmarks (general domains) with enterprise benchmarks (your domain). Option B is a demo, not an eval. Option D is monitoring, which is necessary but different from eval.

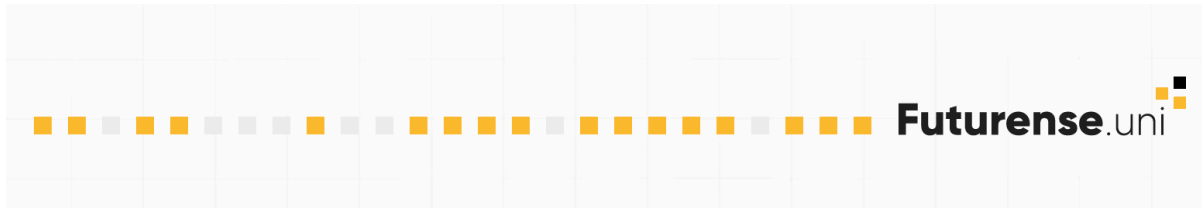
Q5. For a travel itinerary AI, which model characteristic is most important to benchmark?

- A) Counterfactual reasoning: the model must handle hypothetical scenarios.
- B) Correlation: the model must know which routes are popular.
- C) Intervention and planning: the model must reason about what happens if flight times change, hotel availability shifts, or preferences conflict.
- D) Context: the model must handle 200-page travel guides.

Answer: C. Travel planning is an intervention task: given a goal and constraints, plan a sequence of actions, and reason about what happens when one element changes (missed flight, full hotel). Counterfactual is not required. Context is less critical than planning capability.

Q6. A model is 85% accurate but inconsistent (60% consistency). How does confidence gating improve this system?

- A) It increases accuracy to 100% by filtering wrong answers.



- B) It routes low-confidence outputs to fallback (human review or safer responses), so only high-confidence outputs reach customers, improving apparent reliability at the cost of coverage.
- C) It re-runs the model until it gives a consistent answer.
- D) It replaces the model with a more consistent one automatically.

Answer: B. Confidence gating trades coverage for reliability. High-confidence outputs go to customers directly. Low-confidence outputs are intercepted. The model's accuracy on the subset that passes confidence gating is higher than its overall accuracy. Coverage (how many questions the system answers directly) decreases, but reliability (accuracy on answered questions) increases.

Q7. Counterfactual reasoning is not available in current production models. What is the correct design response for an enterprise use case that requires it?

- A) Use a larger model that has counterfactual capability.
- B) Prompt the model with explicit counterfactual instructions and hope for the best.
- C) Redesign the use case to avoid counterfactual reasoning, or route counterfactual questions to human experts with the model providing supporting analysis only.
- D) Inform the customer that AI cannot handle their use case.

Answer: C. No current model reliably handles counterfactuals. The correct response is system design: either restructure the problem to avoid requiring counterfactual conclusions, or build a human-in-the-loop path for counterfactual decisions where the model provides supporting information and a human makes the final call.

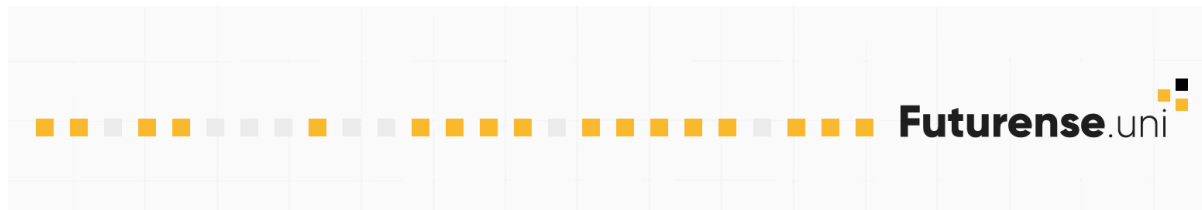
Q8. What is the primary reason not to use published benchmarks (MMLU, HumanEval) as your eval for an enterprise use case?

- A) Published benchmarks are too expensive to run.
- B) Published benchmarks are designed for marketing purposes: they use domains with clear right answers (math, science, code). Enterprise domains have messy inputs, causal relationships, and domain-specific knowledge not represented in these benchmarks.
- C) Published benchmarks are only available for GPT-4 and not other models.
- D) Published benchmarks require proprietary data that cannot be shared.

Answer: B. Published benchmarks optimize for demonstrating model capability in closed domains. Enterprise benchmarks measure your use case on your domain. The gap between these two is precisely where enterprise AI failures occur.

Q9. An FDE recommends using a large expensive model for all tasks in a system. What is the most likely problem with this recommendation?

- A) Large models are less accurate than small models.
- B) Large models are over-engineered and over-expensive for simple tasks (FAQ, lookup) that small models handle equally well, and the unnecessary cost reduces system viability.



- C) Large models cannot handle multiple tasks simultaneously.
- D) There is no problem. Large models are always the best choice.

Answer: B. Model selection should be per task type. Simple FAQ tasks require correlation-level reasoning, which small models handle well at a fraction of the cost. Spending large-model budget on FAQ wastes resources that should be reserved for high-complexity, high-risk decisions that require larger models.

Q10. Read this output: 'correlation: accuracy=80%, consistency=80% / intervention: accuracy=40%, consistency=60% / counterfactual: accuracy=20%, consistency=40%'. For which task types is this model safe to deploy without compensating design?

- A) None — the model requires compensating design for all task types.
- B) Correlation tasks only — where the model is 80% accurate and 80% consistent.
- C) Correlation and intervention tasks only.
- D) All task types — the model exceeds the 50% accuracy threshold.

Answer: B. 80% accuracy and 80% consistency is deployable for low-stakes correlation tasks with a confidence gate for the 20% uncertain cases. Intervention (40% accuracy) requires significant compensating design. Counterfactual (20% accuracy, 40% consistency) should not be deployed for any enterprise decision without full human review.

9.2 Assignments

Assignment 1: Run a Causal Benchmark on Your Problem Domain

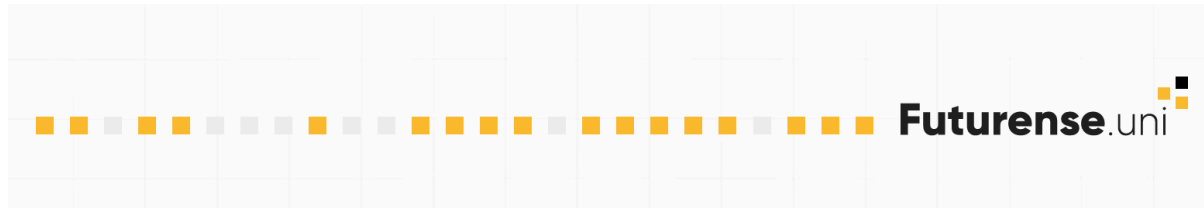
Goal: Apply Pearl's causal hierarchy to a real use case from your domain.

Choose one problem you are currently working on or the problem from your Session 2 assignment.

Requirements:

- Write one correlation-level test question for your domain. What pattern exists that the model might incorrectly infer as causal?
- Write one intervention-level test question. What would happen if a specific variable were deliberately changed?
- Write one counterfactual question. If a specific past decision had been different, what would the outcome have been?
- Run each question 5 times using any available API. Record accuracy (did the answer match the correct reasoning?) and consistency (how often did the same answer appear?).
- Interpret: for which causal level is this model safe for your domain without additional design?

Submit: Six questions/answers (one set per level), a consistency table, and a 150-word interpretation.



Strong answer: Questions are domain-specific, not generic. Interpretation explains which tasks the model handles and which require compensating architecture. Weak answer: Generic questions copied from examples. No interpretation.

Assignment 2: Design a Confidence Gating System

Goal: Apply the confidence gating pattern to a real production scenario.

Scenario: A bank uses AI to assess whether a loan application is complete before routing it to an underwriter. The AI must identify missing documents, incorrect field values, and formatting errors.

Requirements:

- Classify this task by causal level. What level of reasoning is required?
- Define three confidence bands: high (direct to underwriter), medium (flag for review with specific concern noted), low (return to applicant with explanation).
- Write a Python function that calls any LLM API with a structured prompt asking for both the analysis and a confidence score.
- Test on at least 3 example applications (one clean, one with minor issues, one with serious issues).
- Show the routing outcome for each.

Submit: Working Python code with 3 test outputs showing routing decisions. Include the confidence gating thresholds you chose and the rationale.

Assignment 3: Build a Multi-Model Task Map

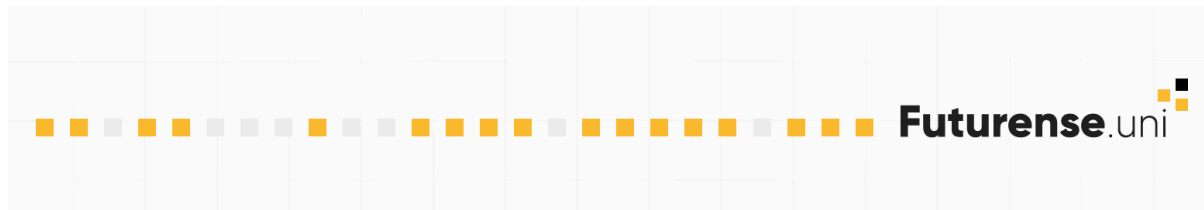
Goal: Apply the eval-first design process to a complete system.

Choose a multi-step AI system (examples: a customer support AI, an HR screening system, a code review assistant, an insurance intake system).

Requirements:

- List all tasks the system must perform. Aim for 5-8 tasks.
- Classify each task by complexity: low (FAQ/lookup), medium (routing/analysis), high (decision/assessment).
- For each high-complexity task, write one test case at each of the three causal levels.
- Based on complexity and causal level, assign a model size recommendation (small, medium, large) with rationale.
- Identify one task that should be routed to human review regardless of model quality, and explain why.

Submit: A task map table plus one paragraph explaining your highest-risk task and the compensating design for it.



10. Further Learning

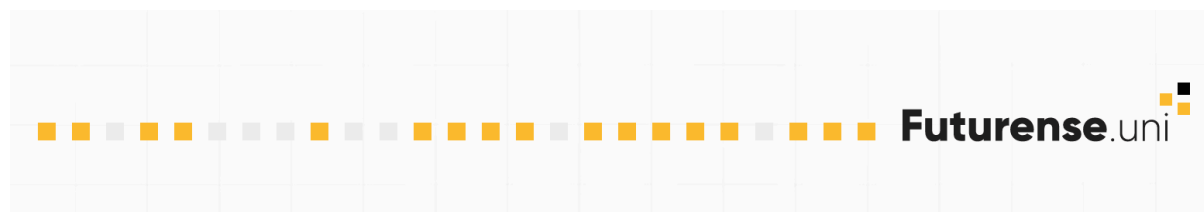
These resources build directly on what was covered in this session.

- Judea Pearl and Dana Mackenzie, 'The Book of Why' (2018). Chapter 1 covers the causal hierarchy in detail. Chapters 8 and 9 apply it to AI. This is the source material for Section 3.1.
- RLVR and reasoning model training: search for 'Reinforcement Learning from Verifiable Rewards DeepSeek' for the original technical paper describing how LRMs are trained for intervention-level reasoning.
- Context window benchmarks: the Needle in a Haystack benchmark (search 'Needle in a Haystack LLM benchmark') is the standard test for context degradation. Run it on any model you are considering.
- Confidence calibration: search 'LLM calibration Platt scaling' for the technical approach to post-hoc confidence calibration of model outputs.
- RAGAS framework: the Retrieval-Augmented Generation Assessment System provides structured eval metrics for RAG pipelines including faithfulness, answer relevance, and context precision.
- Enterprise AI failure analysis: search 'enterprise AI failure rate 87%' for the McKinsey and Gartner research underpinning the 87% failure statistic. Understanding why pilots fail strengthens the case for eval-first design.

11. Cheatsheet

Pearl's Causal Hierarchy

Level	Name	Question	Model Status	Enterprise Risk
1	Correlation	$P(Y X)$ — given X , what is Y ?	Strong in all models	False causation inference
2	Intervention	$P(Y \text{do}(X))$ — if we force X , what happens?	Partial (~20%) in LRMs	Wrong policy predictions



3	Counterfactual	$P(Y_x \text{past})$ — if X had been different?	Unreliable in all models	Hallucinated liability assessments
---	----------------	---	--------------------------	------------------------------------

Test all three levels before deploying any model for enterprise decisions.

Confidence Gating Bands

Band	Confidence	Action
High	>85%	Direct delivery to customer
Medium	65-85%	Deliver with disclosure or offer to review
Low	<65%	Escalate to human expert or return safe fallback

Thresholds are tunable per use case. Higher-stakes decisions require higher thresholds.

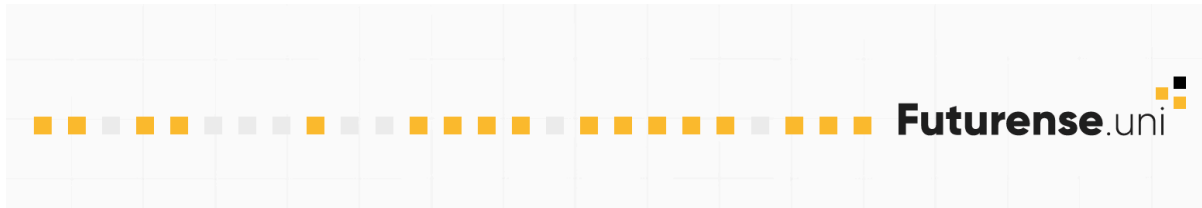
Model Selection by Task Type

Task Type	Causal Level	Model Size	Rationale
FAQ, lookup, Q&A	Correlation	Small (7-13B)	Pattern matching sufficient; low liability
Root cause, routing	Intervention	Medium (30-70B)	Needs causal reasoning; moderate cost
Decisions, assessments	Counterfactual	Large (70B+)	Highest liability; invest in capability
Hybrid system	Mixed	Route by task	Small for FAQ, large for decisions

Never use one model for all tasks. Always allocate by causal level and liability.

Eval Checklist

- Test set: minimum 50 cases per task type, including edge cases and deliberate failures
- Causal levels: test correlation, intervention, and counterfactual separately
- Consistency: run each test 5 times, require >80% agreement for deployment



- Context: test fact placement at 10%, 50%, and 90% of max document length
- Gating criteria: define minimum acceptable score before starting eval
- Compensating design: for every failure mode found, document the guardrail or fallback
- Monitoring: set up live eval metrics to detect drift post-deployment

Formulas

System Reliability = Model Accuracy × Coverage + Fallback Quality × (1 - Coverage)

Consistency Score = max(answer_frequency) / total_samples

Causal Readiness = (Correlation Score × 0.2) + (Intervention Score × 0.4) + (Counterfactual Score × 0.4)

Deployment Gate = Consistency ≥ 0.80 AND Causal Readiness ≥ threshold_for_use_case